

Stock Assist — Project Guide

A simple guide to understand what this project is, how it is built, and how the AI part works. Written for someone who is just starting with React and Node.js.

1. What is Stock Assist?

Stock Assist is a **web app that helps a trader decide what to do with an Indian stock (NSE) or a commodity (Gold, Silver, Crude Oil)**.

You type a stock name like `RELIANCE` . The app:

1. Fetches the live price and past price history of that stock.
2. Reads the latest news about it.
3. Calculates technical indicators (numbers traders use, like RSI and moving averages).
4. Sends all of this to an AI model, which writes an easy explanation.
5. Shows you **two plans**: what to do if the price goes **up** (bullish) and what to do if it goes **down** (bearish) — with entry price, stop-loss and target.

Think of it as a "research assistant" for trading. It does not place real trades.

New to trading words like RSI, stop-loss or support? See the 1-line cheat sheet in **section 12** at the end.

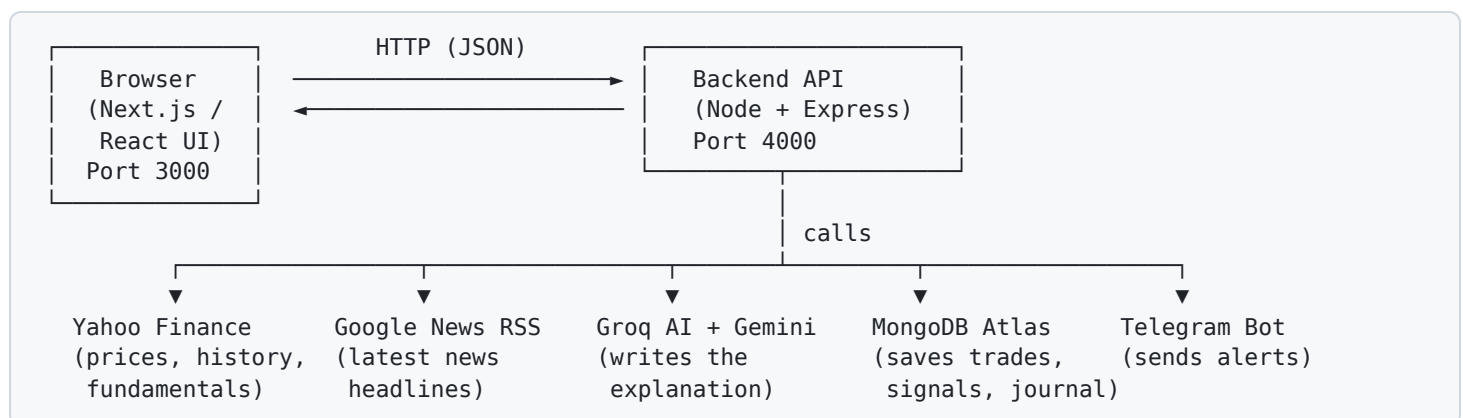
2. Main Features (what the user sees)

Page	What it does
Home (Top Signals)	Scans many NSE stocks and shows the ones with the clearest buy/sell signals.
Scanner / Analyze	Deep analysis of one stock: price chart, indicators, news, AI explanation, bullish + bearish trade plans.
Commodity	Same analysis for Gold, Silver, Crude Oil etc. Shows prices in COMEX (USD), MCX (INR) and Spot.
History	List of past analyses, with filters (stock, date, confidence).
Journal	Your personal trading diary — write notes, and log trades with entry price, exit price and profit/loss.
Backtest	Checks how the app's past signals actually performed ("was the app right?").

Extra features:

- **Hindi + English** — the whole UI and even the AI explanation can switch to Hindi.
- **Dark / Light theme.**
- **Installable app (PWA)** — can be "installed" on a phone like a normal app.
- **Telegram alerts** — sends a message to Telegram when a strong signal is found.
- **Paper trading** — every strong signal automatically becomes a "fake" trade (₹1,00,000 virtual money per trade), so you can see how the app performs without risking real money.

3. Big Picture — How the pieces connect



In simple words:

- **Frontend** = what you see in the browser (buttons, charts, cards).

- **Backend** = a Node.js server that does the heavy work: fetch data, do the math, talk to AI, save to database.
- The frontend **never** talks to Yahoo or AI directly. It only asks the backend, and the backend does everything.

4. Tech Stack (with simple meaning)

Frontend — apps/web

Technology	What it is (simple)	Why we use it
Next.js 14	A framework built on top of React	Gives pages/routing out of the box (<code>app/analyze/page.tsx</code> becomes <code>/analyze</code>)
React 18	Library to build UI using components	Every card, button and chart is a component
TypeScript	JavaScript + types	Catches mistakes before running the code
Tailwind CSS	Write styles using class names like <code>p-4 text-white</code>	Fast styling without separate CSS files
TanStack React Query	Handles API calls, loading states and caching	No need to write <code>useState</code> + <code>useEffect</code> for every API call
Lightweight Charts (by TradingView)	Candlestick stock charts	Shows the price chart
Framer Motion	Animation library	Smooth animations (splash screen, journal)
Lucide React	Icon pack	All the icons
Sonner	Toast popups	"Network error" style messages

Backend — apps/api

Technology	What it is (simple)	Why we use it
Node.js	Runs JavaScript on the server	The backend language
Express	Small framework to create APIs	Defines routes like <code>POST /api/analyze/single</code>
TypeScript	Same as above	Type safety
MongoDB + Mongoose	NoSQL database + a helper library to use it	Stores trades, journal, watchlist, signals
yahoo-finance2	npm package to get stock data from Yahoo	Prices, history, company fundamentals
groq-sdk	Official Groq client	Talk to Llama AI models
@google/generative-ai	Official Google Gemini client	Talk to Gemini AI
Zod	Input validation	Rejects bad requests (e.g. empty stock name)
Helmet, CORS, express-rate-limit	Security middleware	Safe headers, only allowed websites can call the API, limit spam requests
node-cache	In-memory cache	Don't call Yahoo/AI again if we just fetched the same data
Pino	Logger	Clean server logs
Vitest	Testing framework	Unit tests for indicators, scoring etc.

Shared — packages/shared

A small package for code that both apps can use: common **types** (like the shape of a `StockAnalysis` object), **constants** (list of stocks, trading rules) and small **utility functions**. They are written once and reused. The backend uses it the most; the frontend keeps most of its own types in `apps/web/src/types`.

DevOps / Hosting

Tool	Use
npm workspaces (monorepo)	Frontend, backend and shared code live in one repo
GitHub Actions	On every push: type-check, run tests, build the frontend (<code>.github/workflows/ci.yml</code>)
Vercel	Hosts the Next.js frontend (<code>vercel.json</code>)
Render	Hosts the Express backend (<code>render.yaml</code>)

5. Third-Party Services Used

Service	What we get from it	Cost
Yahoo Finance	Live stock price, past price data (daily/weekly/monthly), company fundamentals (P/E, etc.), commodity futures (Gold <code>GC=F</code> , Silver <code>SI=F</code> , Crude <code>CL=F</code>)	Free
Google News RSS	Latest news headlines for a stock, used for news sentiment	Free
Groq	Runs open-source AI models very fast — Llama 3.3 70B (main), Llama 3.1 8B, Mixtral as backups	Free tier
Google Gemini	Google's AI model — Gemini 2.0 Flash	Free tier
MongoDB Atlas	Cloud database	Free tier
Telegram Bot API	Sends signal alerts to a Telegram chat	Free
Vercel / Render	Hosting frontend / backend	Free tier

Every service has a free tier, so the whole project can run at ₹0 cost.

6. How the AI is Implemented (the most important part)

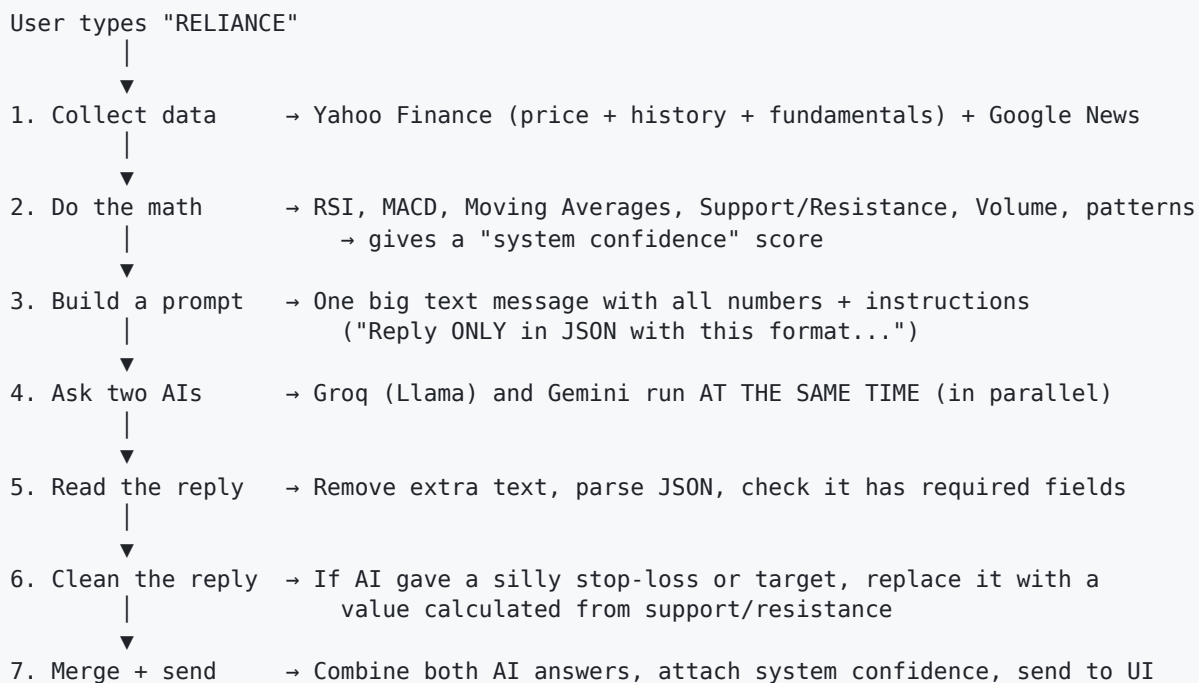
The key idea

Math decides the numbers. AI explains them.

The app does **not** blindly ask the AI "should I buy?". Instead:

- The backend first calculates everything with code (indicators, support/resistance, confidence score).
- Then it gives all these numbers to the AI and asks it to **explain the situation in plain language**, list risks, and describe bullish/bearish scenarios.
- The confidence percentage shown to the user comes from the **system's calculation**, not from the AI. This avoids AI "hallucinating" random numbers.

Step-by-step flow



What is a "prompt"?

A prompt is just a **long text message** sent to the AI. File: `apps/api/src/services/ai/prompt.ts` and `enhancedPrompt.ts`. It looks roughly like this:

You are a professional stock market analyst.

STOCK: RELIANCE PRICE: ₹2,950
RSI: 62 (bullish) MACD: bullish crossover
Support: ₹2,900 Resistance: ₹3,000
News sentiment: positive
...
Reply ONLY with valid JSON in this format:
{ "stock": "...", "bias": "BULLISH | BEARISH | NEUTRAL",
 "bullish": { "trigger": "...", "tradePlan": {...} },
 "bearish": { ... } }

If the user selected Hindi, one extra line is added: *"Write all text in Hindi"*.

Calling the AI (simplified code)

File: `apps/api/src/services/ai/groq.ts`

```
import Groq from 'groq-sdk';

const client = new Groq({ apiKey: process.env.GROQ_API_KEY });

const completion = await client.chat.completions.create({
  model: 'llama-3.3-70b-versatile',
  messages: [
    { role: 'system', content: 'You are a professional stock market analyst. Respond only with valid JSON' },
    { role: 'user', content: prompt },
  ],
  temperature: 0.3, // low = more consistent, less "creative" answers
});

const text = completion.choices[0].message.content;
const analysis = JSON.parse(text); // turn AI text into a JS object
```

Gemini works the same way in `gemini.ts`, using `@google/generative-ai`.

Running two AIs together ("ensemble")

File: `apps/api/src/services/ai/ensembleAI.ts`

```
// Both run at the same time – we don't wait for one to finish before starting the other
const [groqResult, geminiResult] = await Promise.allSettled([
  analyzeWithGroq(input),
  analyzeWithGemini(input),
]);
```

- **Both answered** → merge them, and check if they agree on direction (HIGH / MODERATE / LOW agreement).
- **Only one answered** → use that one.
- **Neither answered** (API down, rate limit, no API key) → use a **fallback** built from pure math. The app never breaks just because AI is unavailable.

`Promise.allSettled` is used instead of `Promise.all` so that if one AI fails, we still get the other one's result.

AI companies retire old models every few months. The model names are written in `services/ai/groq.ts`, `services/ai/gemini.ts` and `services/commodity/index.ts`. If AI explanations stop showing up, check these first — the app keeps working on the fallback, so the problem is easy to miss.

Safety checks on AI output

AI models sometimes return wrong values (e.g. stop-loss = 100 for a ₹2,950 stock). The function `sanitizeScenario` in `singleAnalysisOrchestrator.ts` checks:

- Is the stop-loss on the correct side of the price? (below price for a buy)
- Is it within a sensible range ($\pm 30\%$ of price)?
- If not → replace it with a value calculated from support/resistance.

This is a good talking point in interviews: **"Never trust AI output directly — validate it."**

7. Journey of One Request (end to end)

1. **User** types `RELIANCE` on the Analyze page and clicks Analyze.
2. **React component** calls the custom hook `useAnalysis()` (`apps/web/src/hooks/useAnalysis.ts`).
3. The hook uses **React Query's** `useMutation` which calls `analyzeStock()` in `apps/web/src/services/api.ts`.
4. That function does a `fetch` → `POST http://localhost:4000/api/analyze/single`.
5. On the backend, **Express** receives it. Middleware runs first: security headers, rate limit, Zod validation.
6. The **route** (`apps/api/src/routes/analyze.ts`) hands the work to the **orchestrator** (`services/analysis/singleAnalysisOrchestrator.ts`).
7. The orchestrator fetches data, runs indicators, calls the AI, cleans the output.
8. The result is **saved in MongoDB** (as a signal record + paper trade) and a **Telegram alert** is sent if the signal is strong.
9. JSON response goes back to the browser.
10. React Query stores the data, the component re-renders and shows the chart, cards and trade plans.

8. Folder Structure (where to find things)

```
stock-assist/
├── apps/
│   ├── web/                                ← FRONTEND (Next.js)
│   │   └── src/
│   │       ├── app/                        ← Pages. Each folder = one URL
│   │       │   ├── page.tsx                → "/" (Home / Top Signals)
│   │       │   ├── analyze/               → "/analyze"
│   │       │   ├── commodity/             → "/commodity"
│   │       │   └── history/ journal/ backtest/
│   │       ├── components/                ← Reusable UI pieces (StockCard, StockChart, Navbar...)
│   │       ├── hooks/                    ← Custom hooks (useAnalysis, useTopStocks, useChartData)
│   │       ├── context/                  ← Global state (Theme, Language, Watchlist)
│   │       ├── services/api.ts           ← ALL backend calls live here
│   │       └── utils/translations.ts     ← English + Hindi text
│   └── api/                              ← BACKEND (Express)
│       └── src/
│           ├── index.ts                  ← Server starts here
│           ├── routes/                   ← API endpoints (analyze, trade, journal, backtest...)
│           ├── services/                 ← The real logic
│           │   ├── data/                 → Yahoo Finance calls
│           │   ├── news/                 → Google News
│           │   ├── indicators/            → RSI, MACD, MA, Bollinger, ADX
│           │   ├── patterns/              → Chart pattern detection
│           │   ├── analysis/              → Combines everything, scoring
│           │   ├── ai/                   → Groq, Gemini, prompts
│           │   ├── commodity/             → Gold/Silver/Crude logic
│           │   ├── backtest/              → Checks past signal accuracy
│           │   ├── paperTrading/          → Virtual trades
│           │   └── notifications/         → Telegram
│           ├── models/                   ← MongoDB schemas (Trade, Journal, Watchlist, SignalRecord...)
│           ├── middleware/               ← Security, validation, error handling
│           └── config/                   ← DB connection, env variables, logger
└── packages/shared/                     ← Types & constants used by both apps
```

9. Key Concepts Used (good to revise)

React / Next.js

Concept	Where it is used
Components & props	<code>components/dashboard/StockCard.tsx</code> — one card per stock
Custom hooks	<code>hooks/useAnalysis.ts</code> — keeps API logic out of the UI
Context API (global state)	<code>context/LanguageContext.tsx</code> — switch Hindi/English anywhere
React Query	<code>useQuery</code> for loading data, <code>useMutation</code> for actions like "Analyze"
File-based routing	<code>app/journal/page.tsx</code> automatically becomes <code>/journal</code>
'use client'	Marks components that run in the browser (need clicks, state)

Node.js / Express

Concept	Where it is used
Routes	<code>routes/*.ts</code> — each file = one group of endpoints
Middleware	<code>middleware/</code> — code that runs before every request (security, rate limit, validation)
Service layer	<code>services/</code> — routes stay thin, logic lives here
Mongoose models	<code>models/Trade.ts</code> — defines what a "Trade" looks like in the DB
Environment variables	<code>.env</code> — API keys are never written in code
async / await + Promise.allSettled	Calling many APIs in parallel
Caching	<code>services/cache.ts</code> — news cached 15 min, fundamentals 24 hrs
Graceful fallback	App works even with no database (demo mode) or no AI key

Main API endpoints

Endpoint	Method	Purpose
/api/analyze/single	POST	Analyze one stock
/api/analyze/stocks	GET	Screen many stocks
/api/analyze/history	GET	Past analyses
/api/analyze/commodity	POST	Analyze a commodity
/api/stocks/top-10	GET	Top signals for the Home page
/api/stocks/chart	GET	Chart data for a stock
/api/trade	GET/POST/PUT/DELETE	Trade CRUD
/api/journal	GET/POST/PUT/DELETE	Trade journal CRUD
/api/watchlist	GET/POST/DELETE	Watchlist
/api/backtest/stats	GET	How accurate past signals were
/api/paper-trade/portfolio	GET	Virtual trading portfolio
/api/alerts	GET/POST	Telegram alerts
/health	GET	Is the server alive?

10. How to Run It Locally

Need: Node.js 18+, a free MongoDB Atlas account, free Groq and Gemini API keys.

```
# 1. Install all packages (frontend + backend + shared)
npm install

# 2. Create the env file for the backend
cp .env.example apps/api/.env
```

Fill `apps/api/.env` :

```
MONGODB_URI=your-mongodb-atlas-connection-string
GROQ_API_KEY=your-groq-key           # from console.groq.com
GEMINI_API_KEY=your-gemini-key       # from aistudio.google.com
PORT=4000
# Optional
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=
```

```
# 3. Start both frontend and backend together
npm run dev
```

- Frontend → <http://localhost:3000>
- Backend → <http://localhost:4000/health>

Other commands: `npm test` (run tests), `npm run build` (production build).

Without MongoDB or AI keys the app still starts in **demo mode** — useful for a quick look.

11. For the Resume

Project title

Stock Assist — AI-Powered Stock & Commodity Analysis Platform

Sample resume bullets

- Built a full-stack trading research app using **Next.js, React, TypeScript, Node.js, Express and MongoDB** in a monorepo with a shared types package.
- Integrated **Yahoo Finance** and **Google News** to fetch live prices, historical data, fundamentals and news for NSE stocks and commodities.
- Implemented a **multi-model AI layer (Groq + Google Gemini)** running in parallel, with JSON output validation and a rule-based fallback when AI is unavailable.
- Calculated technical indicators (RSI, MACD, Moving Averages, Bollinger Bands, ADX) and generated **dual bullish/bearish trade plans** with entry, stop-loss and targets.
- Added **backtesting and automatic paper trading** to measure signal accuracy, plus **Telegram alerts** for strong signals.
- Secured the API with **Helmet, CORS, rate limiting and Zod validation**; added in-memory caching, unit tests (**Vitest**), CI with **GitHub Actions**, and deployment setup for **Vercel (frontend) + Render (backend)**.
- Supported **English and Hindi**, dark mode and installable **PWA**.

Two honest tips

- The README mentions a "70% win rate" — that is a **target**, not a proven result. Don't put a win-rate number on the resume unless you have real backtest numbers to show.
- Only write "deployed" if you have a **live link that works**. If you do, put the link on the resume — a working demo beats any bullet point.

Make it your own

Interviewers often ask *"Which part did you build yourself?"* Have a real answer:

1. Run the project on your own laptop (section 10).
2. Read the files mentioned in sections 6 and 7 — that is the heart of the app.
3. Build at least one feature yourself. Beginner-friendly ideas:
 - **Login system** (JWT or NextAuth) — right now there is no login, so every visitor shares the same journal and watchlist.
 - **Email alerts** next to the Telegram alerts.
 - **A "compare two stocks" page** using the existing analyze API.

Interview questions — with short answers

- 1. Why two AI models instead of one?** Reliability: if one is down or hits its free-tier limit, the other still answers. When both answer, we check if they agree on direction — agreement means more trust.
- 2. What if the AI is down or returns wrong data?** `Promise.allSettled` means one failure doesn't break the other. If both fail, a rule-based fallback is used. AI output is always validated: it must be valid JSON with the required fields, and bad stop-loss/target values are replaced with calculated ones.
- 3. Why doesn't the frontend call Yahoo Finance or the AI directly?** API keys must stay secret on the server. The heavy math also belongs on the server, and caching in one place avoids hitting rate limits.
- 4. `Promise.all` vs `Promise.allSettled`?** `Promise.all` fails as soon as one promise fails. `Promise.allSettled` waits for all of them and tells you which succeeded and which failed.
- 5. Why React Query instead of `useEffect` + `fetch`?** It gives loading and error states, caching and retries for free — less code and fewer bugs.
- 6. What is middleware in Express?** A function that runs before the route handler. Here: `helmet` (security headers), `cors` (which websites may call the API), `compression` (gzip), a rate limiter (stops spam), Zod validation (checks input), and an error handler at the end.
- 7. Why MongoDB and not SQL?** The analysis results are big nested JSON objects whose shape changes often. MongoDB stores JSON-like documents directly, and Atlas has a free tier. SQL would also work but needs more table design up front.
- 8. How are API keys kept safe?** They live in a `.env` file on the server, which is listed in `.gitignore` so it never goes to GitHub. In production they are set in the Render dashboard. The browser never sees them.
- 9. What is a monorepo and why a `shared` package?** One repo holds the frontend, backend and shared code, so everything is versioned and built together. The shared package holds types, constants and helpers that are written once and reused.

10. What is caching and TTL? Caching = remembering a result so the next request is instant and we don't call Yahoo/AI again. TTL (time to live) = how long it is remembered — news 15 minutes, fundamentals 24 hours.

11. What would you improve next? Add user login, move the cache to Redis so it works across multiple servers, and add more tests for the API routes.

12. Trading Words — 1-Line Cheat Sheet

Word	Simple meaning
NSE	National Stock Exchange of India — where stocks like RELIANCE are traded
MCX	Multi Commodity Exchange — Indian market for gold, silver, crude oil (prices in ₹)
COMEX	US market for gold and silver (prices in \$)
Bullish / Bearish	Expecting the price to go up / go down
Signal	The app's suggestion: buy, sell or wait
Entry	The price at which you enter the trade
Target	The price at which you book profit
Stop-loss	The price at which you exit to limit your loss
Risk-reward	Possible profit vs possible loss (1:2 = risk ₹1 to make ₹2)
Support / Resistance	Price levels where the price often stops falling / stops rising
RSI	A 0–100 score: above 70 = maybe bought too much, below 30 = maybe sold too much
MACD	Shows whether the price momentum is getting stronger or weaker
Moving Average (MA)	Average price of the last N days — smooths the chart to show the trend
Bollinger Bands	A band around the price showing its "normal" range
ADX	How strong the trend is (not its direction)
Volume	How many shares were traded — moves with high volume are more trustworthy
Candlestick	One bar on the chart showing a day's open, high, low and close price
Fundamentals	The company's financial health — profit, debt, P/E ratio
Backtest	Testing the strategy on past data to see if it would have worked
Paper trading	Practice trading with virtual money